

Corrections

How to Scale Your Model: Exercise Solutions

Siddhartha Bhimireddy

Contents

1	Intro to Rooflines	2
	Question 5	2
2	All About TPUs	3
	Question 2	3
3	Sharded Matmuls	5
	Question 1	5
	Question 7	5
	Question 9	6
5	Training	8
	Question 3	8
7	Inference	11
	Question 2	11
	Question 5	11
	Question 6	13

Chapter 1

Intro to Rooflines

Question 5

The H100 `bfloat16` FLOPs/s are

$$1979 \text{ TFLOPs/s} = 1.979 \times 10^{15} \text{ FLOPs/s}.$$

Without sparsity, this is 9.895×10^{14} FLOPs/s.

The H100 bandwidth is

$$3.35 \text{ TB/s} = 3.35 \times 10^{12} \text{ bytes/s}.$$

The H100 critical intensity is

$$\begin{aligned} B_{\text{crit}} &= \frac{\text{H100 bfloat16 FLOPs/s}}{\text{H100 bandwidth}} \\ &= \frac{9.895 \times 10^{14}}{3.35 \times 10^{12}} \\ &\approx \boxed{295}. \end{aligned}$$

Chapter 2

All About TPUs

Question 2

Full TPUv5e Pod

A full TPUv5e pod has 16×16 chips. Since each slice has $4 \times 2 = 8$ chips per host, we have

$$\frac{256 \text{ chips}}{8 \text{ chips/host}} = \boxed{32 \text{ hosts}}.$$

It has one core per chip, so it has $\boxed{256 \text{ tensor cores}}$.

The total FLOPs/s of the pod are

$$256 \text{ chips} \times 1.97 \times 10^{14} \text{ FLOPs/s} \approx \boxed{5.04 \times 10^{16} \text{ FLOPs/s}}.$$

The total HBM is

$$\begin{aligned} 256 \text{ chips} \times 16 \text{ GB} &= 4096 \text{ GB} \\ &= \boxed{4.096 \text{ TB}}. \end{aligned}$$

Full TPUv5p Pod

A full TPUv5p pod has $16 \times 20 \times 28$ chips. Each slice is now $2 \times 2 \times 1$. The total number of hosts is therefore

$$\begin{aligned} \frac{16 \times 20 \times 28 \text{ chips}}{2 \times 2 \times 1 \text{ chips/host}} &= \frac{8960 \text{ chips}}{4 \text{ chips/host}} \\ &= \boxed{2240 \text{ hosts}}. \end{aligned}$$

There are two cores per chip, so the total number of tensor cores is

$$2 \text{ cores} \times 8960 \text{ chips} = \boxed{17,920 \text{ cores}}.$$

The FLOPs/s of the pod are

$$8960 \text{ chips} \times 4.59 \times 10^{14} \text{ FLOPs/s} = \boxed{4.11 \times 10^{18} \text{ FLOPs/s}}.$$

Now we have 96 GB of HBM per chip, giving

$$96 \text{ GB} \times 8960 \text{ chips} = \boxed{860.160 \text{ TB}}.$$

Chapter 3

Sharded Matmuls

Question 1

We have a mesh $\{X : 4, Y : 8, Z : 2\}$. Since A is only sharded over X and replicated over Y and Z , the ratio of the total number of bytes taken up by A across all chips to the size of one copy is given by YZ , the axes over which A is replicated. This is

$$8 \cdot 2 = \boxed{16}.$$

Question 7

We have $D = 8192$, $F = 32768$, and $B = 128$ in `bfloat16`. Therefore, our total memory footprint is

$$\underbrace{2DF}_{W_{\text{in}}} + \underbrace{2DF}_{W_{\text{out}}} + \underbrace{2BF}_{\substack{\text{temporary activations} \\ \text{(since } BF \gg BD, \text{ this is an upper bound)}}}.$$

This is equal to

$$2 \cdot 2 \cdot 8192 \cdot 32768 + 2 \cdot 128 \cdot 32768 = 1082.1 \text{ MB total.}$$

This will clearly not fit on one TPU, since we are assuming only 300 MB free per TPU. We have to shard four ways. First, let us calculate t_{math} :

$$\begin{aligned} t_{\text{math}} &= \frac{4BDF}{4 \cdot 1.97 \times 10^{14}} \\ &= \frac{4 \cdot 128 \cdot 8192 \cdot 32768}{4 \cdot 1.97 \times 10^{14}} \\ &= 1.74 \times 10^{-4} \text{ s.} \end{aligned}$$

Since M_{XY} in this case is four, this is a good candidate for tensor parallelism; that is,

$$\text{In}[B, D_{XY}] \cdot_D W_{\text{in}}[D, F_{XY}] \cdot_F W_{\text{out}}[F_{XY}, D] \longrightarrow \text{Out}[B, D_{XY}].$$

This requires a ReduceScatter of

$$\text{Out}[B, D]\{XY\} \longrightarrow \text{Out}[B, D_{XY}]$$

and an AllGather of

$$\text{In}[B, D_{XY}] \longrightarrow \text{In}[B, D].$$

Therefore,

$$\begin{aligned} t_{\text{comms}} &= \frac{2BD}{W_{\text{ICI}}} + \frac{2BD}{W_{\text{ICI}}} \\ &= \frac{4BD}{W_{\text{ICI}}}. \end{aligned}$$

Note that a 2×2 TPU mesh has no wraparound, therefore (under the assumption that it is not connected to its diagonal neighbor) it functions logically like a 4×1 mesh with full wraparound (if you imagine flattening the square into a strip, the connections would look like that). So we can use the bidirectional W_{ICI} and treat the mesh as one axis:

$$\begin{aligned} t_{\text{comms}} &= \frac{4 \cdot 128 \cdot 8192}{9.0 \times 10^{10}} \\ &= 4.66 \times 10^{-5} \text{ s}. \end{aligned}$$

Since we have $t_{\text{comms}} < t_{\text{math}}$, we are safely compute bound.

Question 9

Part 1

We are writing the explicit algorithm for

$$A[N, M_X] \cdot B[M_X, K] \longrightarrow C[N, K]$$

using the strategy of multiplying A and B first before AllReducing the result at the end to fully replicate it.

Each individual device contains $A[N, M_X]$ and $B[M_X, K]$. We first multiply $A[N, M_X]$ and $B[M_X, K]$ together locally:

$$C[N, K]\{X\} = \sum_m a_{nm} b_{mk}.$$

Here, m spans over the local values of m . That is, the values of m are bounded by which indices are available to the shard; we compute the summation using each local $A[N, M_X]$ and $B[M_X, K]$ to obtain one $C_{\text{unreduced}}[N, K]$ matrix on each device.

We finish by taking the partial results and AllReducing them to replicate them, giving $C[N, K]$ as desired.

Part 2

We only need to change the AllReduce to a ReduceScatter. Since we are okay with sharding on either axis, we can choose either axis to scatter over; i.e. either of the following,

$$\begin{aligned}[N_X, K] &= \text{ReduceScatter}([N, K]\{U_X\}) \\ [N, K_X] &= \text{ReduceScatter}([N, K]\{U_X\})\end{aligned}$$

Part 3

Since the cost for a ReduceScatter equals that of an AllGather, the difference in algorithms is given by the ratio between N and M . Specifically, we have

$$\text{AllGather}_X(A[N_X, M]) \quad \text{versus} \quad \text{ReduceScatter}_X(C[N, K]\{X\}),$$

which have costs of NM/W_{ICI} and NK/W_{ICI} , respectively. The ratio between the times is given by

$$\frac{NM}{W_{\text{ICI}}} \cdot \frac{W_{\text{ICI}}}{NK} = \frac{M}{K}.$$

When $M/K > 1$, it means the AllGather is costlier. When $M/K < 1$, it means the ReduceScatter is costlier.

Chapter 5

Training

Question 3

Assume that the context length = 32K tokens, $B = 3 \times 10^6$ total tokens per batch, and a TPUv5p slice of size $16 \times 16 \times 16 = 4096$ chips. The weights use `bfloat16`, while the optimizer state uses `float32`.

Part 1: Replicated Model

The TPUv5p has approximately 96 GB of HBM per chip.

The parameters and optimizer state require

$$\begin{aligned} M_{\text{parameters+optimizer}} &= 26 \text{ GB} + 104 \text{ GB} \\ &= 130 \text{ GB}. \end{aligned}$$

Since $130 \text{ GB} > 96 \text{ GB}$, the parameters and optimizer state cannot be replicated on each chip:

Part 2: Pure FSDP

With FSDP, the weights and optimizer state are distributed across the data-parallel shards and gathered or reduced as needed. The parameter and optimizer-state memory per chip is

$$\frac{130 \text{ GB}}{4096} = 0.03 \text{ GB}$$

The per chip activation memory is approximately

$$\begin{aligned}
M_{\text{activations}} &= 2LB(D + 2F) \\
&= 2(40)(3 \times 10^6)(5120 + 2 \cdot 13824) \\
&= 7.86432 \times 10^{12} \text{ bytes} \\
&\approx 7.86 \text{ TB}, \\
M_{\text{activations/chip}} &= \frac{7.86 \text{ TB}}{4096} \\
&\approx 1.92 \text{ GB}.
\end{aligned}$$

This gives a total per chip memory footprint of $1.92 + 0.03 = 1.95$ GB. This comfortably fits into 96 GB TPUv5p memory

Unfortunately, the per-shard batch size is

$$\frac{3 \times 10^6}{4096} \approx 732.$$

Since $732 < 850$ (the compute bound threshold), pure-FSDP configuration is not viable.

Part 3: Combining FSDP and Tensor Parallelism

Using $\alpha = 2550$, $F = 13824$, and $M_x M_y = 2$, the per chip B to remain compute-bound condition is

$$\begin{aligned}
\frac{B}{N} &> \frac{2550^2}{(13824)(2)} \\
&\approx 235.
\end{aligned}$$

Let $N = 4096$ be the total number of chips. Let X be the FSDP dimension and Y be the tensor-parallel dimension, so that $XY = N$. The optimal value of X is

$$\begin{aligned}
X_{\text{opt}} &= \sqrt{\frac{B}{F} \frac{M_x}{M_y} N} \\
&= \sqrt{\frac{3 \times 10^6}{13824} (4096)} \sqrt{\frac{M_x}{M_y}} \\
&\approx 942 \sqrt{\frac{M_x}{M_y}}.
\end{aligned}$$

Note that the same batch-size condition applies regardless of whether $(M_x, M_y) = (1, 2)$ or $(M_x, M_y) = (2, 1)$, because in either case $M_x M_y = 2$. So let $M_x = 2$, $M_y = 1$. Then

$$\begin{aligned} X_{\text{opt}} &\approx 942\sqrt{2} \\ &\approx 1332. \end{aligned}$$

The nearest convenient mesh size is $X = 1024$ and $Y = 4$.

Taking $X = 1024$ and $Y = 4$, we have 1024-way FSDP and 4-way tensor parallelism. The parameter and optimizer-state memory per chip is

$$\frac{130 \text{ GB}}{1024 \cdot 4} = \frac{130 \text{ GB}}{4096}.$$

The activations are also sharded over X and Y , so the activation memory per chip is

$$\frac{7.86 \text{ TB}}{4096} \approx 1.92 \text{ GB}.$$

and the per-device batch size is

$$\begin{aligned} \frac{B}{N} &= \frac{3 \times 10^6}{4096} \\ &\approx 732. \end{aligned}$$

Since $732 > 235$, this configuration is compute bound.

Runtime

The runtime with 40

$$\begin{aligned} t_{\text{ideal}} &= \frac{6 \cdot (3 \times 10^6) \cdot (13 \times 10^9)}{(4.59 \times 10^{14}) \cdot 4096 \cdot 0.4} \\ &\approx 0.30 \text{ s}. \end{aligned}$$

Chapter 7

Inference

Question 2

For a sequence length of 128K, the KV cache requires

$$\begin{aligned} M_{\text{KV}} &= 128\text{K} \cdot 262\text{ KB} \\ &\approx 33.5\text{ GB}. \end{aligned}$$

With 256 GB of total memory and 18.4 GB required for the model parameters, the maximum batch size is

$$\begin{aligned} B_{\text{max}} &= \frac{256\text{ GB} - 18.4\text{ GB}}{33.5\text{ GB}} \\ &\approx 7 \end{aligned}$$

If we dropped the number of KV heads to 1, then we can increase this by a factor of 8; i.e., up to 56

Question 5

Part 1: Parameter Counts

The attention parameter count is

$$\begin{aligned} N_{\text{attention}} &= 2LDH \cdot [N + K] \\ &= 2 \cdot 64 \cdot 4096 \cdot 256 \cdot [32 + 8] \\ &\approx 5.37 \times 10^9. \end{aligned}$$

The embedding and unembedding contain

$$\begin{aligned} N_{\text{embedding}} &= DV \\ &= 4096 \cdot 32128 \\ &\approx 0.131 \times 10^9. \end{aligned}$$

The MLP parameters are

$$\begin{aligned} N_{\text{MLP}} &= L \cdot 3EDF \\ &= 64 \cdot (3 \cdot 16 \cdot 4096 \cdot 16384) \\ &\approx 206.1584 \times 10^9. \end{aligned}$$

Thus, the total parameter count is

$$N_{\text{total}} \approx (206.1584 + 5.37 + 0.131) \cdot 10^9 = \boxed{211.66 \text{ billion}}.$$

Two of the sixteen experts are activated for each token. Therefore, the activated parameter count is

$$\begin{aligned} P_{\text{activated}} &= \frac{206.1584}{8} + 5.37 + 0.131 \\ &\approx \boxed{31.27 \text{ billion}}. \end{aligned}$$

Part 2: Critical Batch Size

Assume that, for a particular batch, all experts are loaded into memory. This requires loading approximately 16 times more parameters than before, while the amount of compute increases by approximately a factor of 2, assuming the MLP dominates.

The original critical batch size was $B_{\text{crit}} = 240$. The new roofline ratio is worse by a factor of $\frac{16}{2} = 8$. Therefore,

$$\begin{aligned} B_{\text{crit,MoE}} &= 8 \cdot 240 \\ &\approx \boxed{1920}. \end{aligned}$$

Part 3: KV Cache

The KV caches are identical to those of the dense model because the MoE architecture does not change the attention portion.

Part 4: FLOPs per Token

A forward pass requires approximately $2 \cdot N \cdot T$. Since only the activated experts contribute to the forward computation and $N_{\text{activated}} \approx 31.27$ billion, we have

$$\begin{aligned} C_{\text{forward}} &= 2 \cdot N_{\text{activated}} \cdot T \\ &= 2 \cdot (31.27 \times 10^9) \cdot T \\ &= \boxed{(62.54 \cdot 10^9) \cdot T}. \end{aligned}$$

Question 6

Part 1

We shard the experts as $[E_Z, D, F_Y]$ and assume the rest of the parameters are sharded MEgatron style over the Y-axis. The time required to load the model is then

$$\begin{aligned} t_{\text{load}} &= \frac{2 \cdot 206.1584 \cdot 10^9}{8.2 \times 10^{11} \cdot 16 \cdot 8} + \frac{2 \cdot (211.66 - 206.1584) \cdot 10^9}{8.2 \times 10^{11} \cdot 8} \\ &\approx \boxed{5.605 \text{ ms}}. \end{aligned}$$

Each chip stores approximately

$$\frac{2 \cdot 206.1584 \cdot 10^9}{16 \cdot 8} + \frac{2 \cdot (211.66 - 206.1584) \cdot 10^9}{8} = 4.597 \cdot 10^9 \text{ bytes}$$

or 4.597 GB of weights. With 16 GB of HBM per chip, this leaves approximately

$$16 \text{ GB} - 4.597 \text{ GB} = 11.403 \text{ GB}$$

available per chip.

Part 2

The model occupies approximately $2 \cdot 211.66 \text{ GB} = 423.32 \text{ GB}$ in total. A single TPUv5e offers 16 GB per chip. The closest clean power-of-two mesh that fits the model is 8×4 . On this mesh, the per-chip weight memory is

$$M_{\text{chip}} = \frac{2 \cdot 206.1584 \cdot 10^9}{8 \cdot 4} + \frac{2 \cdot (211.66 - 206.1584) \cdot 10^9}{4}$$

≈ 15.636 GB.

This tolerates a batch size of up to

$$\begin{aligned} B_{\max} &= \frac{32 \cdot (16 - 15.636) \cdot 10^9}{262144} \\ &\approx 0.045 \text{ M.} \end{aligned}$$